

TECHNICAL DOCUMENTATION

Database Monitor exa

Architecture, components, deployment, APIs and operations

Product	Database Monitor exa (Oracle, Grid Infrastructure / ASM)
Release / build	DB16.52_Monitor_exa
Platform	Windows x64 desktop executable, Python, Flask 3.1
Target databases	Oracle Database 19c — Production, Standby/DR, Reporting, Standalone
Document date	5 October 2026
Audience	Developers, DBAs, infrastructure and security reviewers

Contents

- 1** Introduction and scope
- 2** System architecture
- 3** Technology stack
- 4** Source code layout
- 5** Configuration and data files
- 6** Database connectivity — Thin mode, Thick mode and OCI
- 7** Monitoring engine, polling and caching
- 8** Metric collection reference
- 9** Standby and reporting database monitoring
- 10** ASM / Grid Infrastructure monitoring
- 11** Remote host monitoring over SSH
- 12** Alerting and email (Microsoft 365)
- 13** Archive log cleanup (RMAN)
- 14** Process lifecycle: startup, heartbeat, shutdown
- 15** REST API reference
- 16** Build, packaging and deployment
- 17** Logging and troubleshooting
- 18** Security considerations
- 19** Known limitations and recommendations

About this document

Every technical claim in this guide is drawn directly from the current source code, with the responsible file named inline. Where the application's behavior is subtle or easy to misread from the UI alone, that nuance is called out explicitly rather than simplified away.

1. Introduction and scope

Database Monitor exa is a self-contained, browser-based monitoring console for Oracle 19c databases — Production, Standby/DR, Reporting and Standalone roles — including environments built on Grid Infrastructure and ASM storage. It ships as a Windows executable (`DB16.52_Monitor_exa.exe`) that starts a local Flask web server on `http://127.0.0.1:5001` and opens the dashboard in the default browser.

A DBA loads a list of databases from a CSV/TXT configuration file and then sees, in one place:

- Database and listener availability, uptime and active sessions for every configured database.
- Tablespace usage, SGA/PGA memory, archive log / Fast Recovery Area usage and database growth.
- RMAN backup status and history.
- Sessions: counts, top CPU and memory consumers, blocking locks, with the ability to kill a session.
- Standby (Data Guard) log-gap synchronisation and Reporting database reachability.
- Host-level telemetry over SSH: mount points, Oracle OS processes, and ASM disk group usage via the Grid Infrastructure OS account.
- Email alerts through Microsoft 365 (OAuth2) or SMTP basic authentication.
- Archive log cleanup via RMAN with a configurable retention period.

Scope of this document

This guide describes the internal design, configuration, interfaces, deployment and operational behaviour of the application as implemented in the source code. End-user operation is covered in the companion *User Guide*. Figures in both documents are architecture/flow diagrams derived from the code, not literal screenshots of the running application.

2. System architecture

The application is a single Python process (Flask) serving a server-rendered dashboard and a JSON REST API. All monitoring logic runs inside this process, in a `services/` layer that talks to Oracle databases (SQL*Net), database hosts (SSH) and Microsoft 365 (SMTP/OAuth2). State is kept in memory and in small JSON/CSV files beside the executable — there is no external database or message broker.

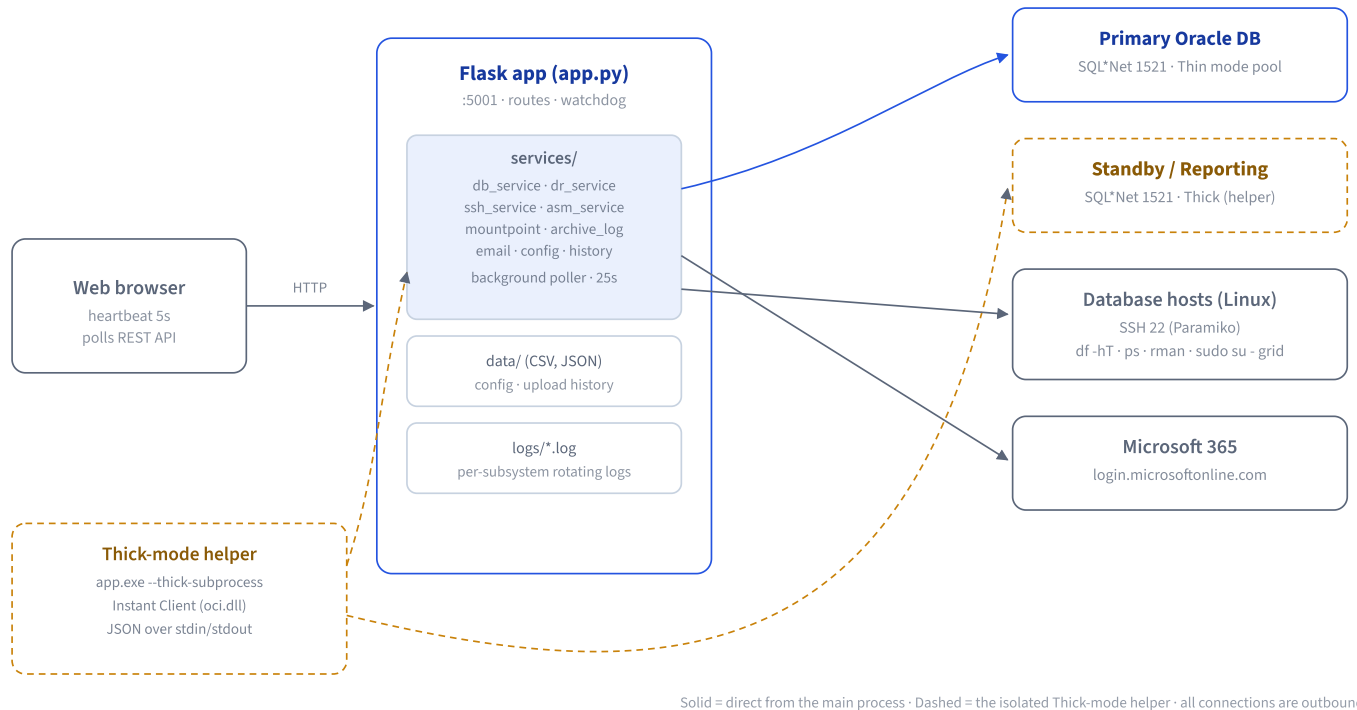


Figure 1 — Component and network architecture.

2.1 Request flow

1. The browser loads the dashboard and polls JSON endpoints (e.g. `/api/db-summary?db_id=...`) on independent timers per card.
2. Most summary data is served from an in-memory cache filled by a background poller (section 7), so requests return immediately even when a database is slow or unreachable.
3. Routes delegate to service functions, which open pooled Oracle connections, pooled SSH sessions, or OAuth2 SMTP sessions as required and return plain dictionaries that Flask serialises to JSON.

2.2 Design principles visible in the code

- **Non-blocking UI:** a database with no cached summary yet returns a "Checking..." placeholder while an async poll runs on a 3-worker thread pool.
- **Process isolation:** Thick-mode Oracle connectivity (needed for Standby/Reporting) runs in a short-lived child process, so the main process stays in Thin mode indefinitely — see section 6.

- **Graceful degradation:** every monitoring-package call has a direct-SQL fallback (section 6.3).
- **Cache invalidation is targeted:** uploading a new configuration invalidates pools, SSH sessions and caches only for the db_ids whose settings actually changed, then pre-warms them on a background thread.

3. Technology stack

LAYER	LIBRARY	VERSION	PURPOSE
Web	Flask, Werkzeug, flask-cors	3.1.3 / 3.1.8 / 6.0.2	HTTP server, routing, templates, CORS
Oracle driver	python- oracledb	4.0.0	Thin mode (main process) and Thick mode (helper subprocess)
Oracle client	Oracle Instant Client	19.31	Bundled under oracle_client/ ; loaded only by the Thick-mode helper
SSH	Paramiko	5.0.0	Remote df , ps , RMAN and sudo su - execution
Email	msal, smtplib	1.37.0 / stdlib	Microsoft Entra ID client- credentials token, SMTP XOAUTH2 / Basic
Host metrics	psutil	7.2.2	Local OS metrics on the machine running the monitor
Crypto	cryptography, bcrypt, PyNaCl	48.0.0 / 5.0.0 / 1.6.2	Dependencies of Paramiko / MSAL
Packaging	PyInstaller	6.20.0	One-folder Windows build
Front end	HTML5, vanilla JavaScript	—	Dashboard UI (static/js/script.js)
Prerequisite	Microsoft Visual C++ Redistributable x64	prerequisites/VC_redist.x64.exe	Required by the bundled Instant Client DLLs

Bundled but unused

The PyInstaller spec collects `azure-identity` and `msgraph-core` in full, but the email code path only imports `msal` and talks to SMTP directly — these two packages add to the build size without being exercised by any code path found in the repository.

4. Source code layout

```
project/
├─ app.py # Flask app: routes, upload parser, watchdog, entry point, -
├─ deploy_pkg.py # Standalone (re)deploy of the PL/SQL monitoring package
├─ pkg_spec.sql / pkg_body.sql # GREENWORLD_MONITOR_PKG source (identical to package_defini
├─ database_procedures.sql # Third, byte-for-byte-consistent copy of the same procedure
├─ requirements.txt
├─ DB16.52_Monitor_exa.spec # PyInstaller build spec (one-folder, console=True)
├─ templates/index.html # Main dashboard UI
├─ templates/backup_graph.html # Backup sessions chart page
├─ static/js/script.js, css/ # Dashboard behaviour, 5s heartbeat, one-shot Auto Refresh
├─ services/
│   ├─ db_service.py # Pools, 25s background poller, PL/SQL package deploy, kill
│   ├─ dr_service.py # Standby/Reporting/Standalone gap checks, Thick-subproces
│   ├─ ssh_service.py # Pooled SSH connections (password / key / Grid user), Aut
│   ├─ remote_process_service.py # Remote ps, orphan-process detection
│   ├─ mountpoint_service.py # Remote df -hT, 60s cache
│   ├─ asm_service.py # ASM disk groups via opc -> sudo su - grid_user -> sysas
│   ├─ archive_log_service.py # Archive/FRA usage + RMAN cleanup with history
│   ├─ listener_service.py # TCP-probe listener status (not lsnrctl)
│   ├─ os_service.py # v$osstat host CPU/mem (not SSH); legacy "disk" = tables
│   ├─ email_service.py # MSAL OAuth2 + SMTP Basic, alert evaluation, history
│   ├─ config_service.py # databases.csv load/save, ASTON/FIAT host rule, cache in
│   ├─ history_service.py # Upload history (restore/delete)
│   └─ package_definition.py # 12-procedure PL/SQL package source, auto-deployed on co
├─ data/databases.csv, *.json # Active config + history (wiped on every GET /, see 5.4)
├─ oracle_client/instantclient_19_31/ # Bundled Instant Client – used only by the Thick helpe
├─ prerequisites/VC_redist.x64.exe
└─ tests/test_config_generation.py # Only config_service.py's parsing/ASTON-FIAT logic is uni
```

4.1 Service module responsibilities

MODULE	OWNS
<code>db_service.py</code>	Connection pooling, the 25s background summary poller, the monitoring-package deploy/fallback pattern, session kill, alert evaluation, <code>graceful_shutdown()</code>
<code>dr_service.py</code>	Standby/Reporting/Standalone status; drives the isolated Thick-mode helper subprocess over stdin/stdout JSON
<code>config_service.py</code>	<code>databases.csv</code> normalisation and save; ASTON/FIAT reporting-host rule; a 3-worker <code>telemetry_executor</code> for async pre-warming
<code>ssh_service.py</code>	Pooled Paramiko sessions for password, private-key and Grid-user authentication; <code>AutoAddPolicy</code> host-key handling
<code>asm_service.py</code>	ASM disk group discovery via the fixed <code>opc</code> SSH login and a <code>sudo su - <grid_user></code> switch
<code>mountpoint_service.py</code>	Remote <code>df -hT</code> parsing, 60s cache per host
<code>remote_process_service.py</code>	Remote <code>ps</code> parsing and orphaned-process flagging
<code>os_service.py</code>	Host CPU/memory from <code>v\$osstat</code> (via the Oracle connection, not SSH)
<code>archive_log_service.py</code>	Archive/FRA usage and RMAN-based retention cleanup, with a persisted history file
<code>email_service.py</code>	MSAL OAuth2 token acquisition, SMTP send (XOAUTH2 or Basic), 17 alert-type evaluation, retry logic
<code>history_service.py</code>	Upload-history records; restore/delete
<code>listener_service.py</code>	TCP reachability probe of the listener port
<code>package_definition.py</code>	The 12-procedure <code>GREENWORLD_MONITOR_PKG</code> source, re-deployed on every new pool

5. Configuration and data files

5.1 Three accepted upload formats

POST /api/upload-databases accepts a .csv or .txt file and auto-detects which of three formats each line uses:

- **Key=value / key:value blocks** — one key per line (detected when a line contains = or : with fewer than 5 commas); many alias spellings are accepted per field.
- **Positional CSV** — comma-separated, parsed with `csv.reader`.
- **Positional whitespace-delimited** — no comma present, split on whitespace.

Both positional forms share the same field layout: 6 required leading fields, up to 21 total (20 positional + an optional trailing `grid_user`). An 'N/C' sentinel at field position 8 disables the Reporting block for that row entirely.

#	FIELD	REQUIRED
1-6	db_id, host, port, service_name, username, password	Yes
7-8	os_user, os_password	Recommended (SSH telemetry, cleanup)
9-16	rep_* (8 fields) — or 'N/C' at position 8 to disable	Optional
17+	stby_* (8), standalone_* (8), grid_user — key=value format only for standby	Optional

5.2 Validation

The upload parser raises explicit, line-numbered errors rather than guessing:

```
Line {n}: Database '{db_id}' is missing required configuration: {fields}.
The uploaded file is the source of truth, so nothing is generated
automatically - supply the full row, or comment this line out with '#' to skip it.

Line {n}: Row is invalid. After 'N/C', at most 6 Standby fields plus an
optional grid_user are allowed.

Line {n}: Row is invalid. Database configuration must contain at most
21 fields (20 plus the optional grid_user).
```

Route-level errors: "No file part in the request", "No selected file", "Only CSV and TXT files are supported", "Uploaded file is empty", "No valid database configurations found in file".

5.3 Reporting host auto-fill (ASTON / FIAT)

If a row supplies `rep_service_name` but leaves `rep_host` blank, the host is filled in automatically from a one-way latch that starts at ASTON and flips permanently to FIAT the first time it sees a `rep_service_name` containing "fiat" (case-insensitive), in file order:

STATE	AUTO-FILLED REP_HOST
ASTON (default, until the trigger)	10.10.4.184
FIAT (from the triggering row onward)	10.10.4.100

An explicitly supplied `rep_host` is never overwritten. This exact rule is exercised by `tests/test_config_generation.py`.

5.4 Configuration is reset on every page load

GET / wipes the active configuration

Every load of the home route — not just the first — clears the active session, calls `save_database_configs([])` and `clear_all_history()`. The configuration file and the upload history are both emptied on every browser refresh. Use the in-page **Refresh** control to update data without losing the loaded configuration; a literal browser reload (F5) requires re-uploading the CSV.

5.5 Files under data/

FILE	WRITTEN BY	CONTENT
databases.csv	config_service	Active configuration, 33 normalised columns
uploads/upload_<ts>_<id>_<name>	history_service	Raw copy of each upload
upload_history.json	history_service	Upload records (id, filename, time, db_ids, status)
email_settings.json	email_service	Global alert/email settings; secrets base64 with an ENC: prefix
email_history.json	email_service	Sent/failed email records
archive_cleanup_history.json	archive_log_service	RMAN cleanup runs incl. output
growth_history.json	db_service	Historic size snapshots used by growth reporting

6. Database connectivity — Thin mode, Thick mode and OCI

This is the single most misread part of the architecture, so it's worth stating precisely: **the main Flask process never loads Oracle's native client**. It connects to every Production database purely in python-oracledb's **Thin mode** — a pure-Python implementation of Oracle's network protocol that needs no Oracle Call Interface (OCI) library at all. The bundled `oracle_client/` folder, the VC++ Redistributable prerequisite, and `oci.dll` exist solely to support a second, deliberately isolated connection path used only for Standby, Reporting and Standalone-DR gap checks.

6.1 Primary databases — Thin mode, pooled

`get_connection()` / `get_connection_by_id()` never call `oracledb.init_oracle_client()`; they only read `oracledb.is_thin_mode()` to log which mode is active. One connection pool is created per `db_id`, lazily, on first use:

SETTING	VALUE	EFFECT
min / max / increment	2 / 30 / 2	2 warm connections at startup, growing 2 at a time to 30 under load
getmode	POOL_GETMODE_WAIT	A request waits for a free connection instead of failing outright
ping_interval	30s	An idle connection is health-checked before reuse
mode	SYSDBA (conditional)	Used automatically, as a standalone (non-pooled) connection, when the configured username is <code>sys</code>

6.2 Standby / Reporting / Standalone — the isolated Thick-mode helper

python-oracledb can only be in one mode per process, and cannot switch after first use — so Thick mode cannot simply be turned on for one query and off for the next. Instead, every Standby, Reporting and Standalone gap check is delegated to a short-lived **child process**, launched as `app.exe --thick-subprocess`, which is the only place in the codebase that calls `init_thick_client()`.

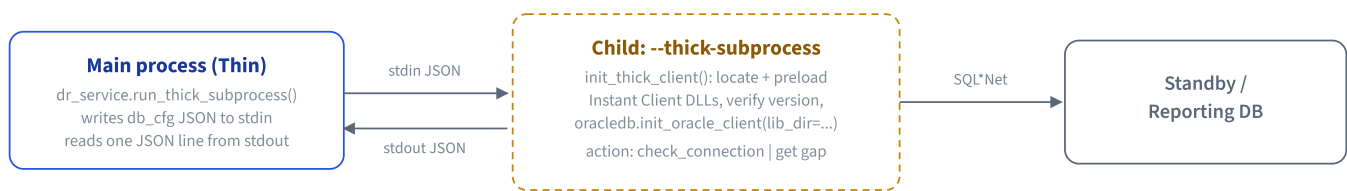


Figure 2 — The Thick-mode helper process used for every Standby, Reporting and Standalone gap check.

On Windows the child is created with `CREATE_NEW_PROCESS_GROUP` so that Ctrl+C in the parent's console does not propagate to it. The parent writes the database's connection details as a JSON object to the child's stdin, reads the first JSON line the child writes to stdout, and treats the exchange as failed if nothing is returned promptly. Missing or mismatched Instant Client components fail loudly rather than silently falling back to Thin mode, since a mismatched `oci.dll` would otherwise produce confusing downstream errors.

6.3 The self-deploying monitoring package

Whenever a new connection pool (or a new SYS standalone connection) is created, the application compiles `GREENWORLD_MONITOR_PKG — AUTHID CURRENT_USER`, 12 procedures, each opening a ref cursor over a fixed query — into the connected schema, from `services/package_definition.py`. This happens once per cold pool, not on every request.

PROCEDURE	PRIMARY VIEW(S)
GET_TABLESPACE_USED	dba_data_files, dba_free_space
GET_LAST_BACKUP / GET_BACKUP_HISTORY / GET_DAILY_BACKUP_STATS / GET_BACKUP_SESSIONS_CHART / GET_SESSION_LOG	v\$rman_backup_job_details
GET_SESSIONS	v\$session, v\$resource_limit
GET_ACTIVE_SESSIONS_CPU	v\$session, v\$sesstat, v\$statname, v\$process, v\$sqlarea
GET_TOP_MEMORY_SESSIONS	v\$session, v\$sesstat, v\$statname, v\$process, v\$sqlarea
GET_DATABASE_GROWTH / GET_DB_GROWTH	dba_tablespaces, dba_tablespace_usage_metrics, dba_data_files, dba_temp_files, v\$database, v\$datafile
GET_MEMORY_INFO	v\$sgainfo, v\$sgastat, v\$pgastat, v\$process_memory

Each metric function tries the package call first and falls back to the equivalent inline direct SQL on any exception — so a connected user missing `CREATE PROCEDURE` privilege still gets working metrics, just without the package's single round-trip efficiency. One metric, **blocking sessions**, has no package procedure at all and is direct-SQL-only. The connected user needs `CREATE PROCEDURE` and read access to the `V$/DBA_` views above; `pkg_spec.sql`, `pkg_body.sql` and `database_procedures.sql` are kept in sync with `package_definition.py` and can be deployed manually with `deploy_pkg.py`.

7. Monitoring engine, polling and caching

Contrary to a naive per-request model, almost every summary figure shown in the UI is served from memory, refreshed by a dedicated background thread — not queried live on each browser poll.

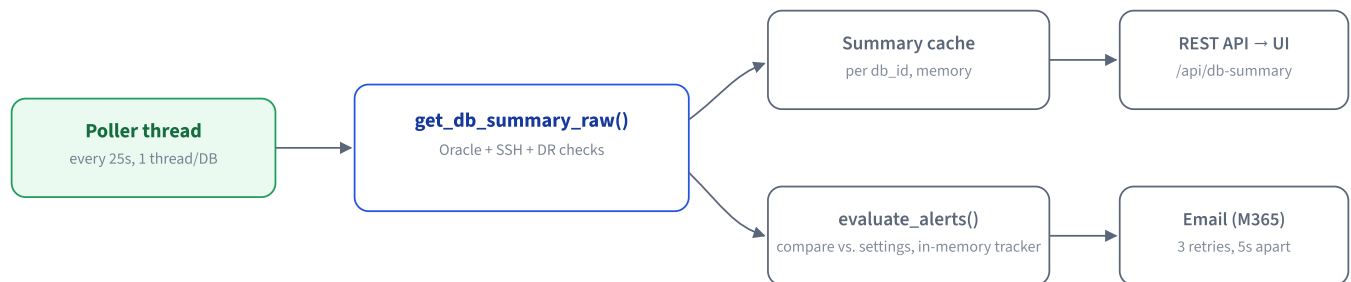


Figure 3 — Background polling, caching and alert evaluation.

CACHE	REFRESH	NOTES
Database summary	Background poll every 25s; 2s startup delay	A never-before-seen <code>db_id</code> returns "Checking..." and is fetched asynchronously via a 3-worker pool (<code>telemetry_executor</code>); an in-flight guard skips a DB whose previous poll hasn't finished
Mount points	60s	One SSH <code>df -hT</code> per unique host
Oracle processes	Cached per host	One SSH <code>ps</code> per unique host
OAuth2 access token	Token lifetime – 300s	Memory only

For the active session's selected database only, `/api/db-summary` additionally runs a cheap pool-reachability check that can override the cached `db_status/listener_status` fields — everything else in the response still comes from the 25-second cache.

8. Metric collection reference

METRIC	SOURCE	NOTES
DB status	Pool/connection acquire success	Not a query — a successful connection <i>is</i> "up"
Uptime / startup time	<code>v\$instance.startup_time</code>	
Listener status	Raw TCP <code>connect_ex()</code> on host:port	Not <code>lsnrctl</code> ; short-circuits to "Running" whenever the DB itself is reachable, without a separate port probe
Tablespace usage	Package, fallback <code>dba_data_files/dba_free_space</code>	
RMAN backups	Package, fallback <code>v\$rman_backup_job_details</code>	Latest / 14-day history / daily stats / session log
Sessions, top CPU, top memory	Package (<code>v\$session</code> , <code>v\$sesstat</code> , <code>v\$process</code> , <code>v\$sqlarea</code>)	
SGA / PGA	Package (<code>v\$sgainfo</code> , <code>v\$sgastat</code> , <code>v\$pgastat</code> , <code>v\$process_memory</code>)	
Archive log / FRA	Direct SQL only — <code>v\$database</code> , <code>v\$archive_dest</code> , <code>v\$parameter</code> , <code>v\$recovery_file_dest</code> , <code>v\$archived_log</code>	No package procedure exists for this metric
Blocking sessions	Direct SQL only — <code>v\$session</code> self-join on <code>blocking_session</code>	No package procedure exists for this metric
Database growth	Package (<code>dba_tablespaces</code> , <code>dba_data_files</code> , <code>dba_temp_files</code> , <code>v\$datafile</code>)	
Host CPU / memory	<code>v\$osstat</code>	Via the Oracle connection itself, not SSH
Mount points	SSH <code>df -hT</code>	
Oracle OS processes	<code>ps -eo pid,user,rss,cputimes,vsz,comm,args -no-headers --sort=-cputimes head -200</code>	Over SSH
ASM disk groups	SSH as <code>opc</code> → <code>sudo su - grid_user</code> → <code>sqlplus / as sysasm</code>	See section 10

Naming collision

`os_service.py`'s "disk" figure is derived from `dba_tablespace_usage_metrics` — i.e. it is tablespace usage dressed up as a disk metric, not host filesystem usage. Genuine host disk usage comes only from the separate `df -hT` path in `mountpoint_service.py`. Do not confuse the two when reading alert rules keyed on "disk".

9. Standby and reporting database monitoring

9.1 Reporting database

`get_reporting_status(db_id)` returns `Not Configured` when `rep_db_id/rep_host` is empty; otherwise the Thick helper attempts a login and returns `UP` or `DOWN` with the underlying error text.

9.2 Standby (Data Guard) synchronisation — per redo thread

Unlike a single-number gap check, this project computes the gap **per THREAD#** — which matters for RAC primaries with multiple redo threads — and surfaces the worst thread as the headline figure:

```
SELECT THREAD#,
       MAX(SEQUENCE#)                                AS LAST_RECEIVED,
       MAX(CASE WHEN APPLIED = 'YES' THEN SEQUENCE# END) AS LAST_APPLIED,
       MAX(SEQUENCE#) - MAX(CASE WHEN APPLIED='YES' THEN SEQUENCE# END) AS LOG_GAP
FROM   V$ARCHIVED_LOG
GROUP BY THREAD#
```

Run against the primary (generated sequence) and, via the Thick helper, against the standby (received/applied). `LOG_GAP > 1` on any thread marks that thread (and the overall database) `Not Synced`; the thread with the worst gap stands in for the single `archive_gap/applied_sequence` figure shown on the summary card. `LOG_GAP ≤ 1` across every thread is `Synced`.

The same per-thread query also runs, internally only, against configured **Standalone** databases purely for logging (one line per thread to `reporting_db.log`) — it never surfaces to the UI or affects that database's displayed status, and is explicitly independent of the Standby/Reporting gap check used elsewhere.

10. ASM / Grid Infrastructure monitoring

ASM disk group discovery has **no configuration of its own** — it reuses the Production row's host from `databases.csv` and follows a fixed three-hop path:

- 1 SSH to the Production host as the `opc` — `os_user` / `oracle` , which is reserved for mount points and process listing.
fixed OS account never
- 2 From `sudo -n su` — passwordless sudo already `grid_user` column is blank, it `"grid"` .
that - configured on the host, not a defaults to the literal
session, `<grid_user>` stored credential. If the optional OS username
run
- 3 As the Grid account, run `sqlplus / as sysasm` and query `v$asm_diskgroup` .

Why there's no password column for `grid_user`

Grid/ASM authenticates entirely differently from every other widget in this application: no DB credential and no separate OS password are ever used — just an OS *username* to switch into, after the `opc` SSH login has already authenticated. A failure on this path can never mark a database DOWN or disturb the Oracle/SSH pools used elsewhere; it is fully isolated.

The `opc` account name is the default compute-instance OS user on Oracle Cloud Infrastructure (OCI) — this module's fixed login identity is a strong signal the tool targets Oracle hosts provisioned on OCI (Exadata Cloud Service / DB Systems) rather than arbitrary on-premises Linux boxes, even though nothing in the code hard-codes an Exadata-specific API.

11. Remote host monitoring over SSH

- SSH uses the row's `os_user/os_password` (or key, where configured) for mount points and process listing; the separate `opc` identity is used only for ASM (section 10).
- Host key verification uses `paramiko.AutoAddPolicy()` on every SSH path (password, key, and Grid user) — see the security note in section 18.
- Databases sharing a host are grouped so only one SSH command and one result block is produced per host, not one per database row.

FEATURE	COMMAND
Mount points	<code>df -hT</code>
Oracle OS processes	<code>ps -eo pid,user,rss,cputimes,vsz,comm,args --no-headers --sort=-cputimes head -200</code>
ASM disk groups	<code>sudo -n su - <grid_user> -c "sqlplus / as sysasm ..."</code>
Archive cleanup	Generated shell script running rman

12. Alerting and email (Microsoft 365)

12.1 OAuth2 send flow

```
msal.ConfidentialClientApplication(  
    client_id, client_credential=client_secret,  
    authority=f"https://login.microsoftonline.com/{tenant_id}")  
  
token = app.acquire_token_for_client(  
    scopes=["https://outlook.office365.com/.default"])  
  
SMTP smtp.office365.com:587 -> EHLO -> STARTTLS -> AUTH XOAUTH2 (RFC 7628)
```

The access token is cached in memory only, for its lifetime minus a 300-second safety buffer. SMTP Basic Authentication (plain username/password) is also supported as an alternative to OAuth2.

TLS validation is a toggle, not a guarantee

A setting can disable certificate validation entirely (`ssl._create_unverified_context()`) for both the OAuth2 and Basic-auth SMTP paths. Leave TLS validation enabled in production.

12.2 Alert types and evaluation

17 alert conditions are evaluated against each poll's cached summary: database down, listener down, tablespace > 90%, disk > 90% (tablespace-derived, see section 8's naming-collision note), CPU > 90%, memory > 90%, RMAN backup failed, archive log full, blocking sessions, database startup, database shutdown, standby down, standby not synced, standby log gap, standby destination error, reporting DB down, and mount-point critical — plus free-form custom rules matched against summary keys and an arbitrary threshold.

Frequency: "once until resolved" (re-arms only once the condition clears), or a fixed repeat of 15/30/60 minutes.

Retry: failed sends retry up to 3 times, 5 seconds apart (both configurable).

Alert state does not survive a restart

The "already sent" tracker for once-until-resolved alerts is an in-memory dictionary only. Restarting the application — which also happens on every auto-shutdown — silently clears it, so a condition that was already alerted on can trigger a duplicate email after restart even though it never actually cleared.

Email secrets (`client_secret`, SMTP password) are stored in `data/email_settings.json` as `"ENC:" + base64(value)` — obfuscation, not encryption; trivially reversible by anyone who can read the file.

13. Archive log cleanup (RMAN)

- 1 GET /api/archive-cleanup/info?retention_days=N reads log mode, archive destination, FRA usage and counts logs eligible for cleanup:

```
WHERE COMPLETION_TIME <
SYSDATE - :retention_days AND
DELETED = 'NO'
```
- 2 POST /api/archive-cleanup/run with {"retention_days": N} — default 10, refused outside ARCHIVELOG mode.
- 3 Over SSH, the tool runs RMAN:

```
CROSSCHECK ARCHIVELOG ALL; DELETE NOPROMPT EXPIRED ARCHIVELOG ALL;
DELETE NOPROMPT ARCHIVELOG ALL COMPLETED BEFORE 'SYSDATE-N';
```
- 4 Deleted count, duration and RMAN output are returned and appended to archive_cleanup_history.json .

Destructive, and the shell command isn't fully escaped

Deleted archive logs cannot be recovered — confirm backups (and standby application) before running a cleanup. Separately: the generated remote shell script interpolates the database name, username and password directly into the command text with only a partial sed-based escape applied after the fact, not a proper `shlex.quote()`-style escape before interpolation. A password or service name containing shell metacharacters (`"`, ```, `$()`) could break out of the intended quoting. The values come from the uploaded configuration file, not direct user input, but that file should be treated as a trust boundary, not as inert data. See section 18.

14. Process lifecycle: startup, heartbeat, shutdown

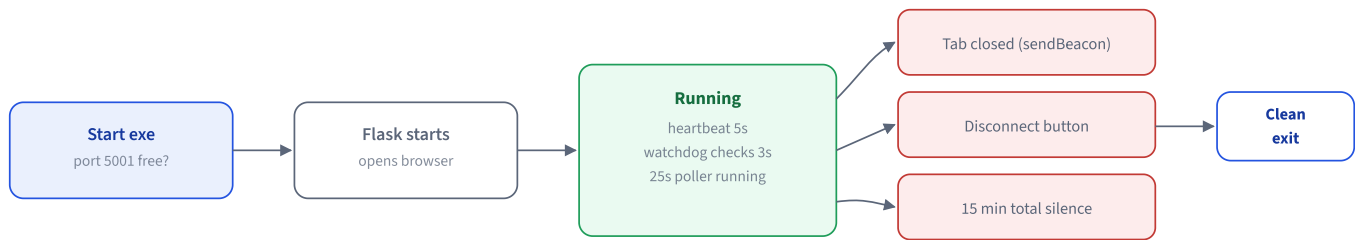


Figure 4 — Application lifecycle and automatic shutdown.

- **Startup:** a pre-flight `socket.bind()` check on port 5001 — on failure, prints a boxed startup error and blocks on `input()` waiting for Enter rather than auto-selecting another port or exiting silently.
- **Frozen (PyInstaller) mode:** `templates/static` resolve under `sys._MEIPASS`; `data/` and `logs/` resolve next to the `.exe` instead of next to `app.py`, and `databases.csv` self-seeds from a bundled default if missing.
- **Heartbeat:** the frontend posts to `/api/heartbeat` every 5 seconds with `keepalive: true`, and sends `navigator.sendBeacon` to `/api/dashboard-closed` on unload.
- **Shutdown triggers:** a close signal followed by 5 seconds without a new heartbeat (a page refresh's new heartbeat cancels it); the **Disconnect** button; or 15 minutes of total silence as a long-tail fallback for a crash that never sent a close signal. All three converge on `graceful_shutdown()`, which closes every Oracle pool and SSH session before `os._exit(0)`.

15. REST API reference

All endpoints are served on `http://127.0.0.1:5001` and return JSON unless noted.

15.1 Application and configuration

METHOD	ENDPOINT	DESCRIPTION
GET	/	Main UI — also wipes config + history (section 5.4)
GET	/dashboard	Legacy dashboard page
GET	/backup-graph	RMAN backup sessions chart page
GET/POST	/api/heartbeat	Keep-alive, resets the close-signal timer
POST	/api/dashboard-closed	sendBeacon tab-closed signal (204)
POST	/api/shutdown	Stop the server immediately
GET	/api/config-status	Uploaded / Refreshed indicator
GET	/api/databases	Configured databases + active db_id — see the exposure note in section 18
POST	/api/set-database, /api/clear-database, /api/delete-database	Select / clear / remove a database
POST	/api/upload-databases	multipart file (.csv/.txt) — section 5
GET/POST/DELETE	/api/upload-history, /api/upload-history/restore, /api/upload-history/<id>	Upload history list / restore / delete

15.2 Database and backup metrics

METHOD	ENDPOINT	DESCRIPTION
GET	/db-status, /listener-status	Active DB / listener status
GET	/api/db-summary?db_id=	Cached health summary — section 7
GET	/tablespace-used, /api/db-growth	Tablespace list, growth projection
GET	/api/rman/latest, /history, /daily-stats, /sessions-chart, /session-log	RMAN backup data
GET	/api/servers-metrics	Combined per-host metrics (parallel)

15.3 Sessions and memory

METHOD	ENDPOINT	DESCRIPTION
GET	/api/sessions/count, /old-count, /cpu-usage, /cpu-usage-24h, /memory-usage, /blocking	Session views
POST	/api/sessions/kill	{"sid","serial"} → ALTER SYSTEM KILL SESSION '<sid>,<serial>' IMMEDIATE
GET	/api/memory-info	SGA/PGA breakdown

15.4 Archive logs, hosts and ASM

METHOD	ENDPOINT	DESCRIPTION
GET	/api/archive-log-info, /api/archive-cleanup/info	Usage / cleanup preview
POST	/api/archive-cleanup/run	Run RMAN cleanup — section 13
GET	/api/oracle-processes, /api/server/all-oracle-processes, /api/server-processes	Oracle OS processes
GET	/api/server/mount-points, /api/server/all-mount-points	Mount points
GET	/api/server/all-asm-diskgroups	ASM disk group usage — section 10
GET	/api/os-info, /api/top-processes	Host CPU/memory, local processes

15.5 Email

METHOD	ENDPOINT	DESCRIPTION
GET/POST	/api/email/config (alias /api/email-settings)	Read (masked) / save settings
POST	/api/email/connect, /disconnect	Validate / clear Microsoft 365 connection
POST	/api/email/test (alias /api/test-email), /api/test-smtp	Send test email
GET/POST	/api/email-history, /api/clear-email-history	Sent email history

16. Build, packaging and deployment

16.1 Prerequisites

- Windows 10/11 or Windows Server, x64.
- Microsoft Visual C++ Redistributable x64 (installer in prerequisites/) — required only for the Thick-mode helper's Instant Client.
- Network access to each database host on 1521 (SQL*Net) and 22 (SSH); to Microsoft 365 endpoints if email alerting is used.
- A database account with SELECT on the views in section 8, CREATE PROCEDURE for the monitoring package, and ALTER SYSTEM if session kill is used. SYSDBA applies automatically when the configured user is sys.
- An OS account on each host for SSH telemetry (os_user), and separately the fixed opc account with passwordless sudo to the Grid user, for ASM.

16.2 Running from source

```
python -m venv .venv-1
.venv-1\Scripts\activate
pip install -r requirements.txt
python app.py # serves http://127.0.0.1:5001 and opens the browser
```

16.3 Building the executable

```
pyinstaller DB16.52_Monitor_exa.spec
# output: dist\DB16.52_Monitor_exa\DB16.52_Monitor_exa.exe (+ _internal\)
```

The spec bundles templates, static, data and oracle_client; collects everything for oracledb, cryptography, msal, paramiko, azure.identity and msgraph_core (the latter two unused, see section 3); builds a **one-folder, console-visible** application (console=True) with UPX compression on both the EXE and the collected folder.

16.4 Deployment

- 1 Install the Visual C++ Redistributable x64 on the target PC.
- 2 Copy the DB16.52_Monitor_exa folder to a directory the user data/ and logs/ are created whole can write to — beside the exe.
- 3 Run the exe. A console window stays open and shows progress; the browser http://127.0.0.1:5001 opens at

4

Upload the database configuration file and, optionally, configure email alert settings.

17. Logging and troubleshooting

LOG FILE (LOGS/)	CONTENT
dashboard_monitor.log	Startup banner, HTTP requests, general errors, shutdown
production_db.log	Primary DB connection attempts and pool events — includes plaintext passwords by design , see section 18
standby_db.log / reporting_db.log	Thick-helper standby/reporting checks, incl. per-thread gap logging for Standalone DBs
asm_grid.log	ASM/Grid Infrastructure SSH queries
email_alerts.log	Email sending and OAuth events

SYMPTOM	LIKELY CAUSE	ACTION
Databases disappear after a refresh	Home-page reset, by design (5.4)	Re-upload the configuration file; use the in-page Refresh instead of F5
Standby/Reporting DOWN with a DLL error	Instant Client or VC++ runtime missing	Install VC_redist.x64; keep oracle_client/ intact; check reporting_db.log
ASM shows unavailable	opc SSH login failed, or passwordless sudo to the Grid user isn't configured	Verify opc connectivity and <code>sudo -n su - grid</code> on the host; check asm_grid.log
Mount points / processes: "Connection Failed"	SSH blocked or wrong os_user/os_password	Check port 22 and the credentials for that row
Duplicate alert emails for an unresolved issue	Alert tracker reset by a restart (12.2)	Expected behavior — not a bug to chase
Test email fails	SMTP AUTH disabled, wrong tenant/client secret, or TLS mismatch	Re-verify Entra ID app registration and mailbox SMTP permission

18. Security considerations

The application holds privileged database and OS credentials and binds only to `127.0.0.1`, so the API is not reachable from other machines — but anyone with access to the workstation account can read everything beside the executable. The following were identified directly in this codebase:

#	OBSERVATION	RECOMMENDATION
1	Plaintext passwords are written to <code>logs/production_db.log</code> on every connection attempt — a code comment explicitly acknowledges this and names the file as sensitive.	Remove credentials from the log statement; rotate any password that has already been logged; restrict filesystem access to <code>logs/</code> .
2	<code>GET /api/databases</code> returns <code>os_password</code> and <code>rep_password</code> to the browser in plaintext, while the function's own docstring says it excludes sensitive fields.	Strip all password fields from this response; this looks like an unintentional gap rather than a deliberate design choice.
3	SSH accepts any host key (<code>paramiko.AutoAddPolicy()</code>) on all three SSH paths.	Pin or verify host keys against a <code>known_hosts</code> file for a trusted network path.
4	Archive cleanup interpolates the database name/credentials into a generated shell script with only partial escaping applied after the fact (section 13).	Escape with <code>shlex.quote()</code> before interpolation, or prefer OS-authenticated <code>rman target /</code> only.
5	The Flask session secret key is a hardcoded literal string, with a code comment telling the reader to change it.	Generate a random key at startup; session cookies are currently forgeable by anyone who reads the source.
6	CORS is wide open (<code>CORS(app)</code> with no origin restriction).	Restrict to the local origin, or remove flask-cors entirely if nothing cross-origin is actually needed.
7	Email secrets are base64-"obfuscated" (<code>ENC:</code> prefix), not encrypted; TLS certificate validation can be switched off entirely from the UI.	Use OS-level credential storage (DPAPI/Credential Manager); keep TLS validation enforced.

19. Known limitations and recommendations

- **Configuration resets on every page load** (5.4) — not just the first; a plain browser refresh, not only a restart, clears the loaded databases.
- **Auto Refresh is one-shot** — the header toggle refreshes once after the chosen delay and then unticks itself, despite reading like a continuous-refresh switch; the individual cards still refresh on their own fixed timers regardless.
- **"Disk" naming collision** — the Disk > 90% alert and `os_service.py`'s disk figure are tablespace usage, not host filesystem usage (section 8); real disk usage lives only in the mount-points metric.
- **Listener status is a TCP probe**, not `lsnrctl status` — a port that merely accepts a TCP connection reads as "Running" even if the listener process itself is unhealthy, and a successful DB connection reports "Running" without even probing the port separately.
- **Alert state resets on restart** (12.2) — a once-until-resolved alert can duplicate-fire after an auto-shutdown/restart cycle for a condition that was never actually resolved.
- **Standby/Reporting require the Production host to be reachable** — both are queried by SSH-tunneling through Production, so a Production outage also blanks out Standby/Reporting visibility even if those databases are themselves healthy.
- **Grid/ASM assumes the opc convention** — on a host where the Oracle-provisioning OS account isn't literally named `opc`, ASM discovery will fail with no fallback identity.
- **Test coverage is narrow** — the only test file, `tests/test_config_generation.py`, covers `config_service.py`'s parsing and the ASTON/FIAT rule exclusively; there are no automated tests for `app.py` routes, `db_service.py`, SSH services, email/alerting, or the Thick-mode helper.
- **Unused packaged dependencies** — `azure-identity` and `msgraph-core` are bundled into every build but not imported anywhere in the email code path (section 3).

End of Technical Guide.